AIML 231 – Week 6

Regression

Dr Qi Chen

School of Engineering and Computer Science

Victoria University of Wellington

Qi.Chen@vuw.ac.nz

Classification vs. Regression

Both are supervised machine learning tasks

- Classification tasks have *discrete* (categorical) output.
  - E.g., *spam* vs *not spam.*
- Regression tasks have *continuous* output.
  - E.g., *house price, temperature.*

- Learning Process:
  - Split the dataset into *training* and *test* sets.
  - Train a *classification algorithm/regression algorithm* on the training set to obtain *classifier/regressor.*
  - Evaluate the *classifier/regressor* on the test set to obtain testing performance.

- Classification and regression have different performance metrics.

House price?

Floor space	Rooms	Lot size	Apartment	Price
71	2	90	1	350,000
98	3	123	0	600,000
128	4	150	0	750,000
86	3	95	1	450,000
173	5	220	0	930,000

Features Label

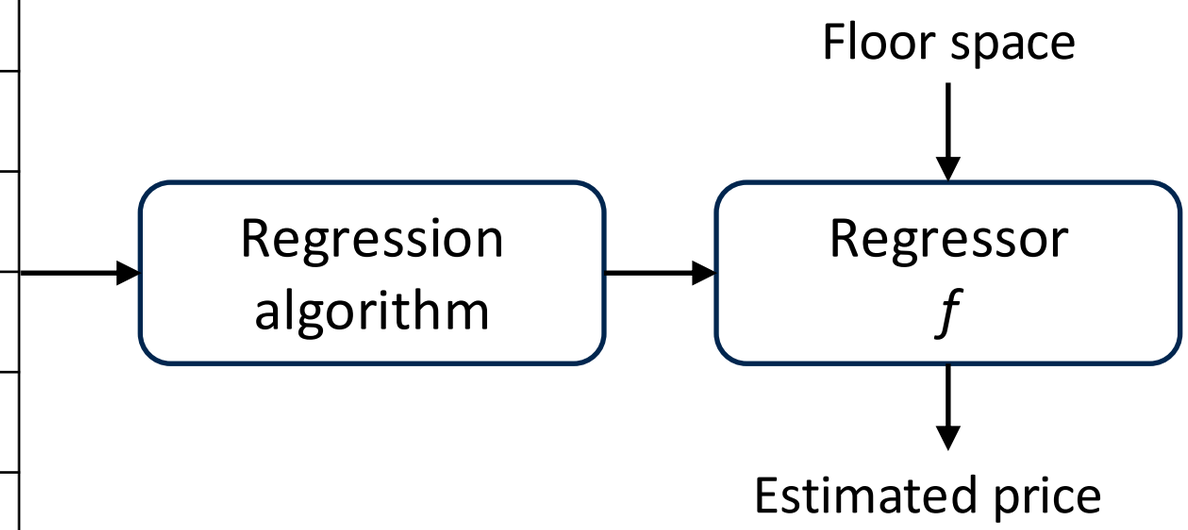
Regression is a supervised learning task that models the relationship between input features and a continuous output variable.

- Features: also known as *independent variables/attributes/ predictors*.
  - E.g., Floor space, Rooms, Lot size…
- Labels: also known as *dependent variables/outputs.*
  - E.g., Price

Simplified House Price

Training set

Floor space	Price
71	350,000
98	600,000
128	750,000
86	450,000
173	930,000



- One input variable: *Floor space*.
- One output label: *House price*.
- Train a regression algorithm on the training set to obtain a regressor.
- The regressor is a function f that takes a floor space of a house, and outputs the *estimated price* of the house.

- How to represent f?

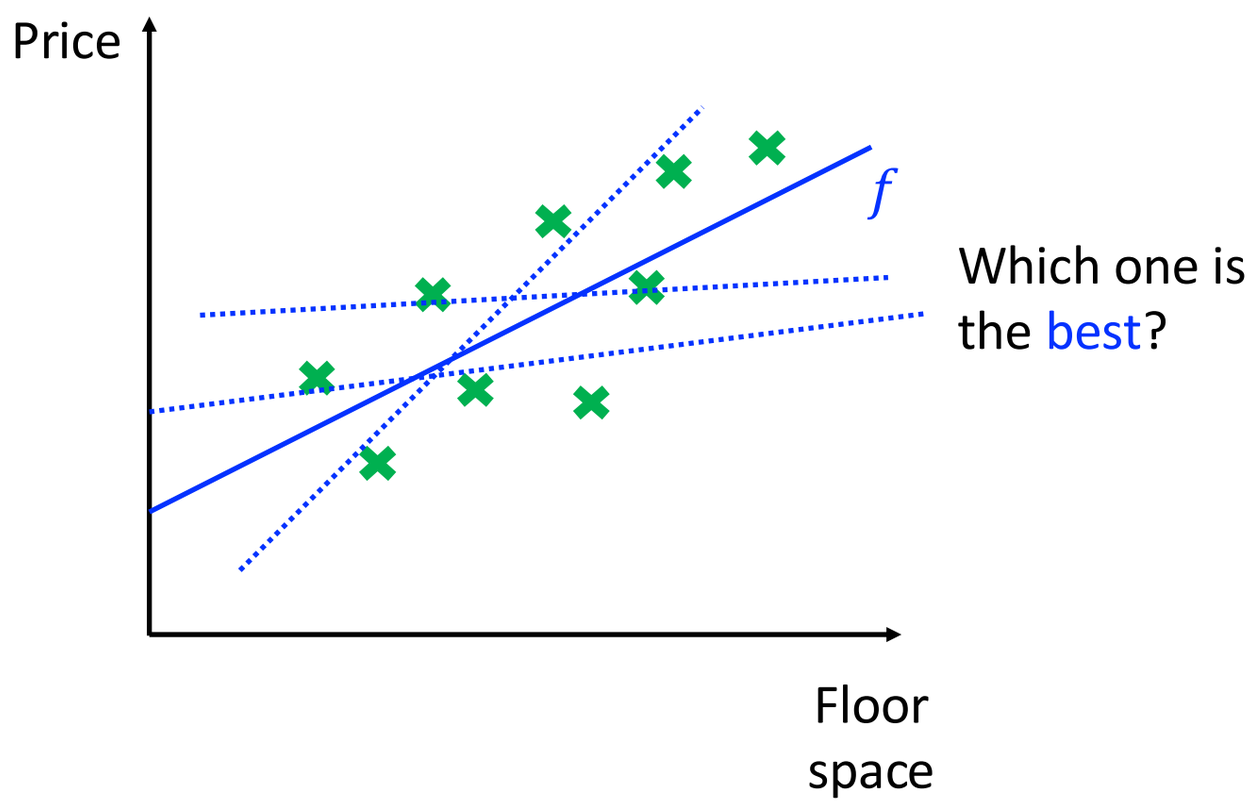
Simple Linear Regression

- **Assumption**: There is only one feature in the regression task

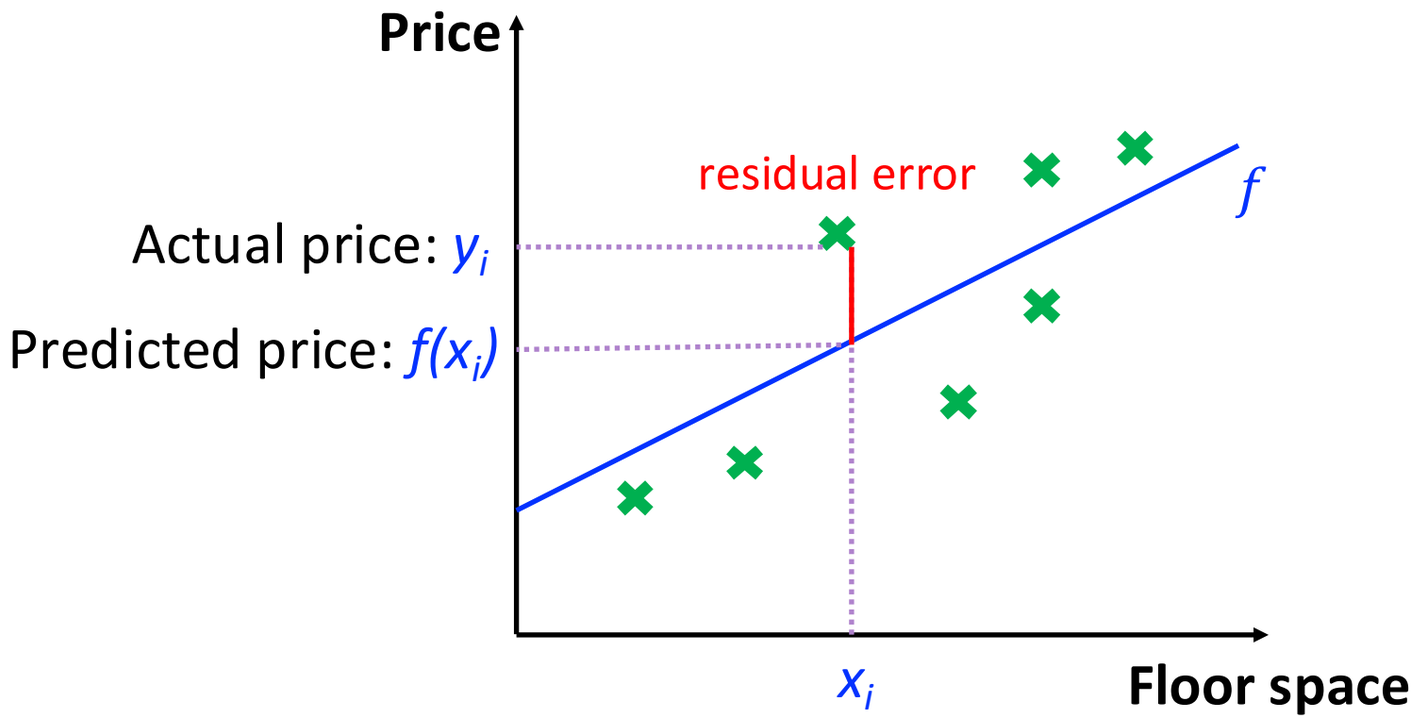
$$Price = f(FloorSpace)$$
$$= w_0 + w_1 \times FloorSpace$$

  - The regressor is a straight line where w_0 is the *intercept* and w_1 is the *slope.*
    - If *FloorSpace* is increased by 1, *Price* is increased by w_1.

  - Linear regression finds the intercept and the slope that *best models* the training data.



Residual Error

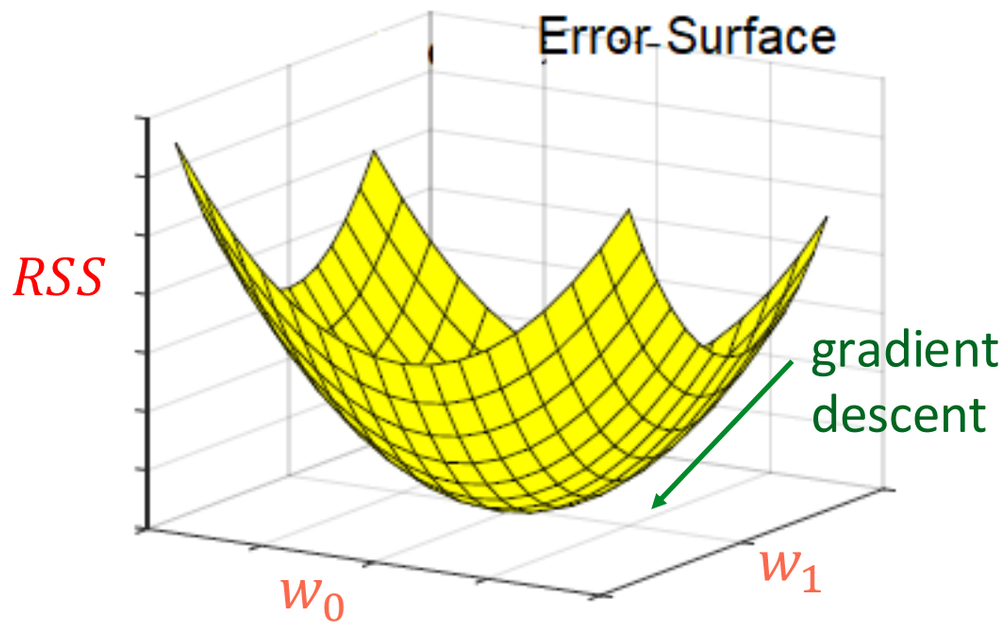


- For the *i*th instance (house), the Residual Error is: $y_i - f(x_i) = y_i - (w_0 + w_1 \times x_i)$
- We usually use the Squared Residual Error $(y_i - f(x_i))^2$ since it is non-negative.
- For *all n* instances in the training set, the error is given by a *sum (Residual Sum of Squares – RSS):*

$$RSS = \sum_{i=1}^{n} (y_i - f(x_i))^2$$

- The best f should minimize RSS.

How to find the optimal *f*?



- Different (w_0, w_1) results in different RSS values, which form the error landscape.
- We can apply Least Squares Estimation (LSE) or Gradient Descent to find out (w_0, w_1) that minimises RSS.
- More gradient descent on Week 8 and LSE in AIML 232!
- *sklearn.linear_model.LinearRegression*

Multiple Linear Regression

Floor space	Rooms	Lot size	Apartment	Price
71	2	90	1	350,000
98	3	123	0	600,000
128	4	150	0	750,000
86	3	95	1	450,000
173	5	220	0	930,000

- Add more *slopes* for more features:

$$EstimatedPrice = w_0 + w_1 \times FloorSpace + w_2 \times Rooms + w_3 \times LotSize + w_4 \times Apartment$$

- Generally, given *d* features, the regressor has one intercept and *d* slopes --- *(d+1)* weights*:*

$$f(x_i) = w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} = w_0 + \sum_{k=1}^{d} w_k x_{ik}$$

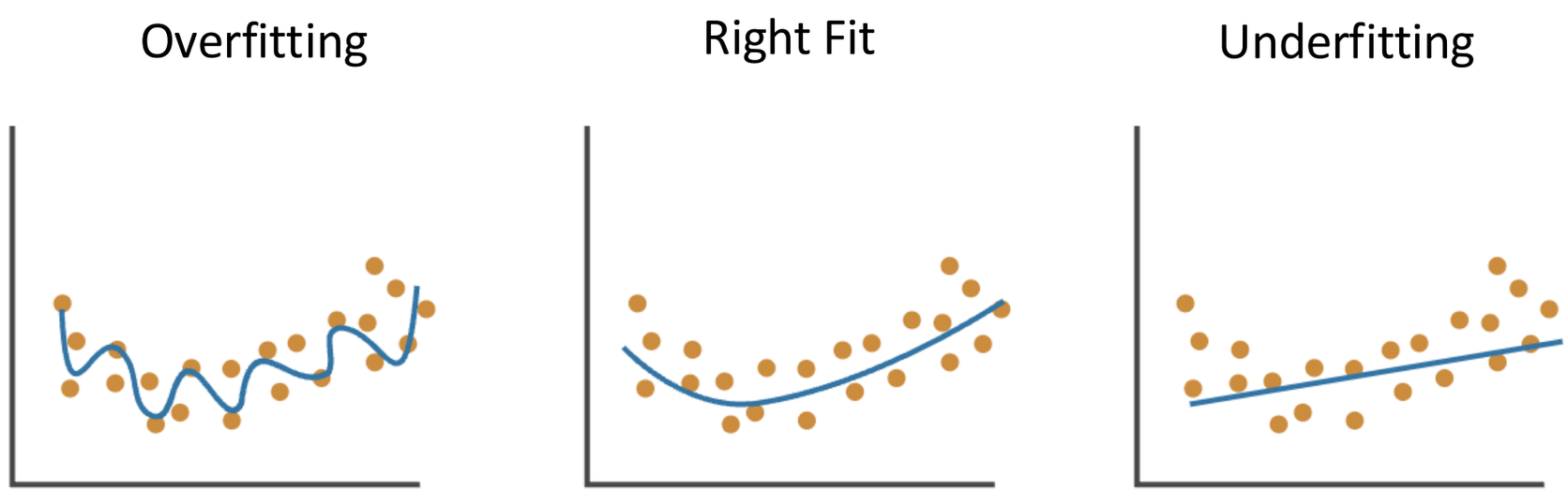
- We still optimise $RSS = \sum_{i=1}^{n} (y_i - f(x_i))^2$

Comments on Linear Regression

- Advantages:
  - Simple and efficient: computationally inexpensive, making it suitable for large datasets.
  - Interpretable: especially useful for explaining the contributions of each feature.

- Disadvantages:
  - Assumes linear relationship.
  - Sensitive to outliers.

Overfitting



- Models are complex with large weights, that are not generalized.

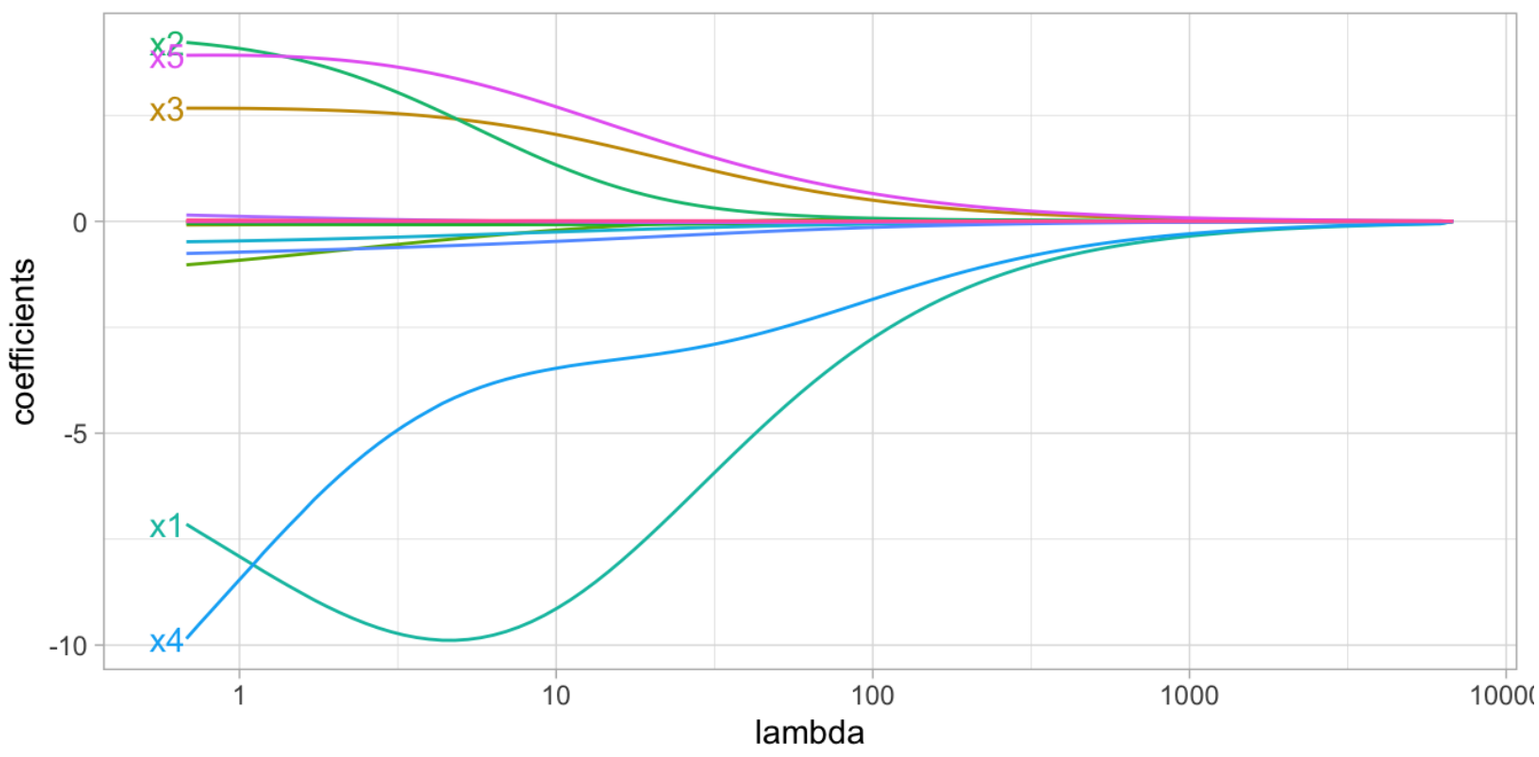
- Regularisation: a technique to penalise large weights, to avoid overfitting.
  - Ridge regression with ℓ_2- regularisation.
  - Lasso regression with ℓ_1- regularisation.

Ridge regression

Adds a penalty based on the sum of the **squared** values of the *slopes*.

$$\text{RSS} + \lambda \sum_{k=1}^{d} w_k^2$$

- λ controls the contribution of the regularisation
- Effect: shrinks the coefficients towards zero but does not set them exactly to zero.



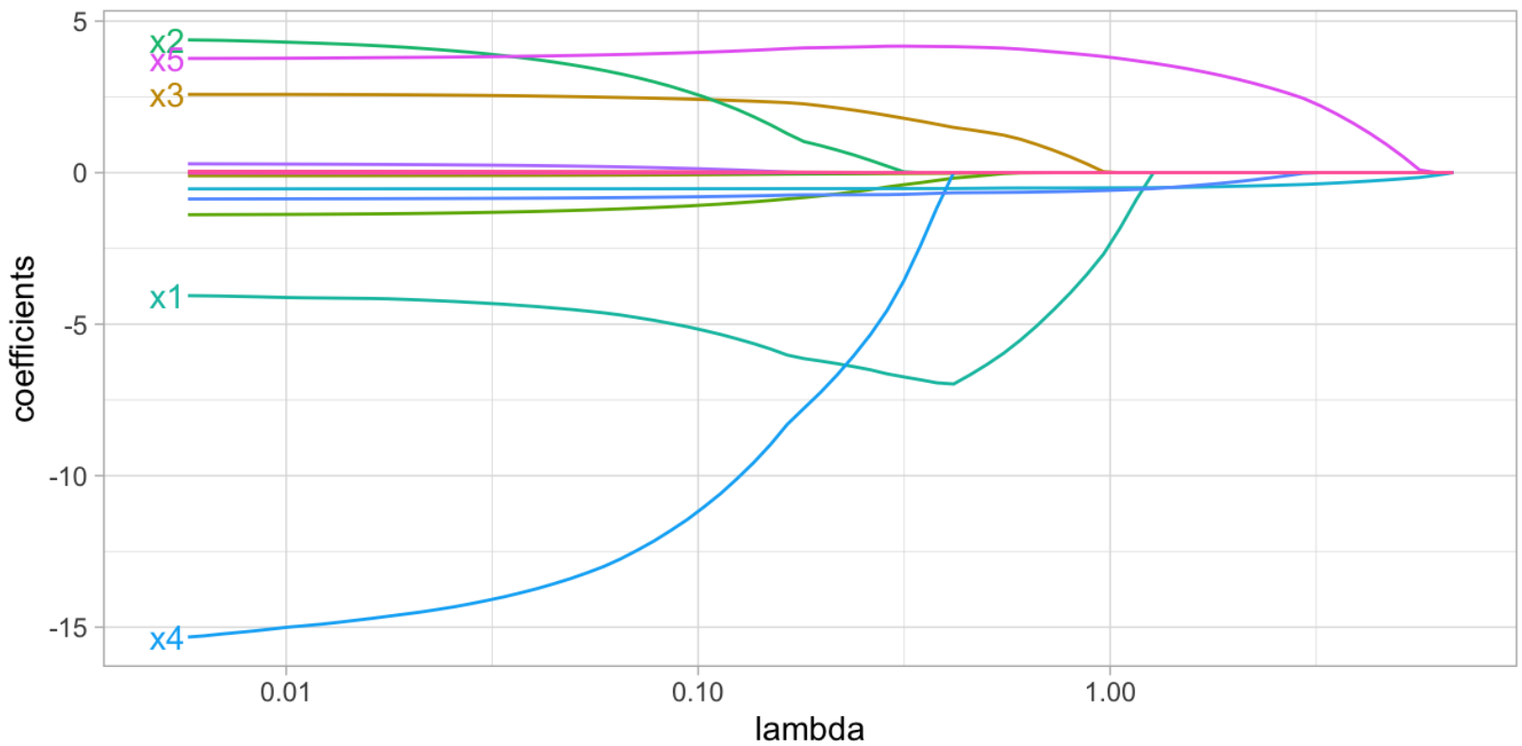
- *sklearn.linear_model.Ridge*

Lasso regression

Adds a penalty based on the sum of the **absolute** values of the *slopes*.

$$\text{RSS} + \lambda \sum_{k=1}^{d} |w_k|$$

- Effect: can drive some weights to zero, effectively removing less important features (embedded feature selection).



- *sklearn.linear_model.Lasso*

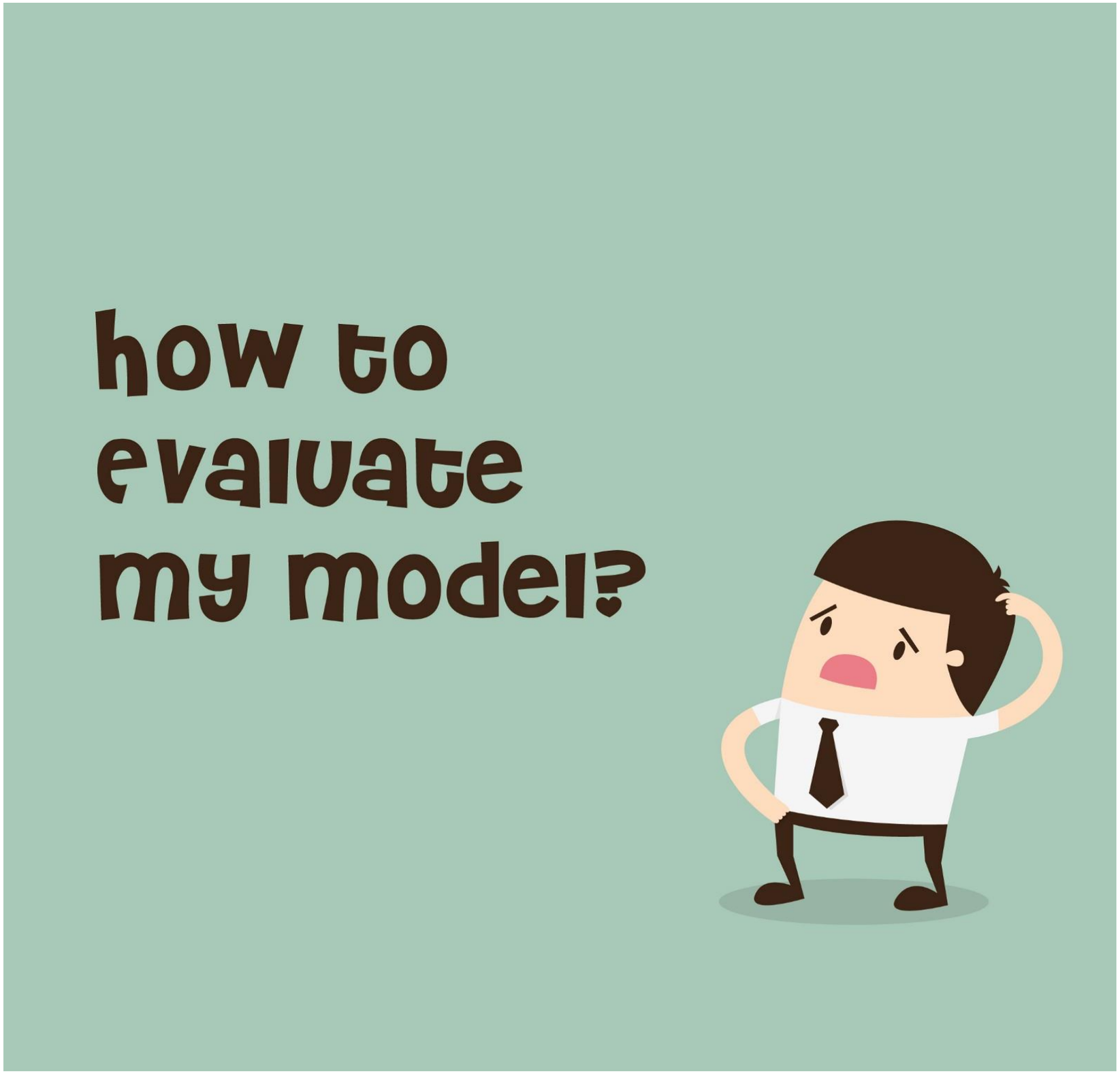
Non-linear Regression (bonus)

- Polynomial Regression
- Gaussian Process Regression
- Exponential Growth Regression
- Logistic Growth Regression

….

- Genetic Programming: no model assumption 😍

Thinking



https://becominghuman.ai/introduction-to-evaluation-metrics-e1254b81a1e8

Mean Squared Error

- Mean Squared Error (MSE)

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \widehat{y_i})^2$$

y_i is the *actual/desired output* of the *i*th instance,

$\widehat{y_i}$ is the *predicted output* of the *i*th instance.

- Lowers MSE indicates better performance.
- MSE is *differentiable.*
- MSE penalises *larger errors* more heavily than *smaller ones* since the *error differences* are squared.

- Sensitive to outliers.
- Not interpretable in original units: MSE is in squared units of the target label.

Mean Absolute Error

- Mean Absolute Error

$$MAE = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}_i|$$

- Less sensitive to outliers than MSE.
- Same scale as the output label.

- Not differentiable at Zero: harder to use in gradient descent.

Root Mean Squared Error

- Root Mean Squared Error (RMSE)
  - Aims to make the scale of errors to be the same scale of target.
  - $RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2} = \sqrt{MSE}$

- Connection to MSE:

$$\boxed{MSE(a) > MSE(b) \iff RMSE(a) > RMSE(b)}$$

- Sensitive to outliers.

R-squared (R^2)

- R-squared (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y}_i)^2} = 1 - RSE$$

  - $\bar{y}_i = \frac{1}{N}\sum_{i=1}^{N} y_i$: simple predictor is to use $\bar{y}_i$ as the predicted value.
  - takes the total squared error and normalizes it by the total squared error of the simple predictor.
  - compare between models whose errors are measured in the different units.
- $R^2 \leq 1$: the higher, the better
  - $R^2 = 1$: perfect fit.
  - $R^2 = 0$: the mxodel is not better than simply predicting the mean.
  - $R^2 < 0$: very poor models.

- Most of the time, we recommend to use R^2.